SemiGuard AI - Backend Code

백엔드 전체 소스 코드

생성일: Fri Aug 7 11:22:54 UTC 2026

파일: server/_core/index.ts

```
import "dotenv/config";
import express from "express";
import { createServer } from "http";
import net from "net";
import { createExpressMiddleware } from "@trpc/server/adapters/express";
import { registerOAuthRoutes } from "./oauth";
import { registerSocialOAuthRoutes } from "./socialOAuth";
import { registerStorageProxy } from "./storageProxy";
import { appRouter } from "../routers";
import { createContext } from "./context";
import { serveStatic, setupVite } from "./vite";

function isPortAvailable(port: number): Promise<boolean> {
  return new Promise(resolve => {
    const server = net.createServer();
    server.listen(port, () => {
      server.close(() => resolve(true));
    });
    server.on("error", () => resolve(false));
  });
}

async function startServer() {
  const app = express();
  const server = createServer(app);
  // Configure body parser with larger size limit for file uploads
  app.use(express.json({ limit: "50mb" }));
  app.use(express.urlencoded({ limit: "50mb", extended: true }));
  registerStorageProxy(app);
  registerOAuthRoutes(app);
  registerSocialOAuthRoutes(app);
  // tRPC API
  app.use(
    "/api/trpc",
    createExpressMiddleware({
      router: appRouter,
      createContext,
    })
  );
  // development mode uses Vite, production mode uses static files
  if (process.env.NODE_ENV === "development") {
```

```
    await setupVite(app, server);
  } else {
    serveStatic(app);
  }

  const port = parseInt(process.env.PORT || "3000");

  server.listen(port, () => {
    console.log(`Server running on http://localhost:${port}/`);
  });
}

startServer().catch(console.error);
```

파일: server/_core/context.ts

```ts
import type { CreateExpressContextOptions } from "@trpc/server/adapters/ex
import type { User } from "../../drizzle/schema";
import { sdk } from "./sdk";

export type TrpcContext = {
  req: CreateExpressContextOptions["req"];
  res: CreateExpressContextOptions["res"];
  user: User | null;
};

export async function createContext(
  opts: CreateExpressContextOptions
): Promise<TrpcContext> {
  let user: User | null = null;

  try {
    user = await sdk.authenticateRequest(opts.req);
  } catch (error) {
    // Authentication is optional for public procedures.
    user = null;
  }

  return {
    req: opts.req,
    res: opts.res,
    user,
  };
}
```

파일: server/_core/trpc.ts

```
import { NOT_ADMIN_ERR_MSG, UNAUTHED_ERR_MSG } from '@shared/const';
import { initTRPC, TRPCError } from "@trpc/server";
import superjson from "superjson";
import type { TrpcContext } from "./context";

const t = initTRPC.context<TrpcContext>().create({
  transformer: superjson,
});

export const router = t.router;
export const publicProcedure = t.procedure;

const requireUser = t.middleware(async opts => {
  const { ctx, next } = opts;

  if (!ctx.user) {
    throw new TRPCError({ code: "UNAUTHORIZED", message: UNAUTHED_ERR_MSG
  }

  return next({
    ctx: {
      ...ctx,
      user: ctx.user,
    },
  });
});

export const protectedProcedure = t.procedure.use(requireUser);

export const adminProcedure = t.procedure.use(
  t.middleware(async opts => {
    const { ctx, next } = opts;

    if (!ctx.user || ctx.user.role !== 'admin') {
      throw new TRPCError({ code: "FORBIDDEN", message: NOT_ADMIN_ERR_MSG
    }

    return next({
      ctx: {
        ...ctx,
        user: ctx.user,
```

```
      },
    });
  }),
);
```

파일: server/_core/oauth.ts

```
import { COOKIE_NAME, ONE_YEAR_MS, OAUTH_STATE_COOKIE, decodeOAuthState }
import { parse as parseCookieHeader } from "cookie";
import type { Express, Request, Response } from "express";
import * as db from "../db";
import { getSessionCookieOptions } from "./cookies";
import { sdk } from "./sdk";

function getQueryParam(req: Request, key: string): string | undefined {
  const value = req.query[key];
  return typeof value === "string" ? value : undefined;
}

export function registerOAuthRoutes(app: Express) {
  app.get("/api/oauth/callback", async (req: Request, res: Response) => {
    const code = getQueryParam(req, "code");
    const state = getQueryParam(req, "state");

    if (!code || !state) {
      res.status(400).json({ error: "code and state are required" });
      return;
    }

    // CSRF guard: the nonce in `state` must match the one-time cookie tha
    // startLogin set in the browser that began this login. An attacker ca
    // forge `state`, but cannot plant this cookie in the victim's browse
    const { nonce } = decodeOAuthState(state);
    const expectedNonce = parseCookieHeader(req.headers.cookie ?? "")[OAU
    if (!nonce || nonce !== expectedNonce) {
      res.status(403).json({ error: "invalid oauth state" });
      return;
    }
    res.clearCookie(OAUTH_STATE_COOKIE, { path: "/", secure: true, sameSi

    try {
      const tokenResponse = await sdk.exchangeCodeForToken(code, state);
      const userInfo = await sdk.getUserInfo(tokenResponse.accessToken);

      if (!userInfo.openId) {
        res.status(400).json({ error: "openId missing from user info" });
        return;
      }
```

```
      await db.upsertUser({
        openId: userInfo.openId,
        name: userInfo.name || null,
        email: userInfo.email ?? null,
        loginMethod: userInfo.loginMethod ?? userInfo.platform ?? null,
        lastSignedIn: new Date(),
      });

      const sessionToken = await sdk.createSessionToken(userInfo.openId,
        name: userInfo.name || "",
        expiresInMs: ONE_YEAR_MS,
      });

      const cookieOptions = getSessionCookieOptions(req);
      res.cookie(COOKIE_NAME, sessionToken, { ...cookieOptions, maxAge: O

      res.redirect(302, "/");
    } catch (error) {
      console.error("[OAuth] Callback failed", error);
      res.status(500).json({ error: "OAuth callback failed" });
    }
  });
}
```

파일: server/_core/socialOAuth.ts

```
import { COOKIE_NAME, ONE_YEAR_MS, OAUTH_STATE_COOKIE } from "@shared/cons
import { parse as parseCookieHeader } from "cookie";
import type { Express, Request, Response } from "express";
import axios from "axios";
import * as db from "../db";
import { getSessionCookieOptions } from "./cookies";
import { sdk } from "./sdk";
import { ENV } from "./env";

interface SocialUserInfo {
  id: string;
  email?: string;
  name?: string;
  provider: "google" | "naver" | "kakao";
}

interface GoogleTokenResponse {
  access_token: string;
  token_type: string;
  expires_in: number;
  id_token?: string;
}

interface GoogleUserInfo {
  sub: string;
  email: string;
  name: string;
  picture?: string;
}

interface NaverTokenResponse {
  access_token: string;
  token_type: string;
  expires_in: number;
  refresh_token?: string;
}

interface NaverUserInfo {
  resultcode: string;
  message: string;
  response: {
```

```
    id: string;
    nickname: string;
    name: string;
    email: string;
    profile_image?: string;
  };
}

interface KakaoTokenResponse {
  access_token: string;
  token_type: string;
  expires_in: number;
  refresh_token?: string;
  refresh_token_expires_in?: number;
}

interface KakaoUserInfo {
  id: number;
  kakao_account: {
    profile_nickname?: string;
    email?: string;
    name?: string;
  };
}

function getQueryParam(req: Request, key: string): string | undefined {
  const value = req.query[key];
  return typeof value === "string" ? value : undefined;
}

async function getGoogleUserInfo(accessToken: string): Promise<GoogleUser
  const response = await axios.get("https://www.googleapis.com/oauth2/v2/
    headers: { Authorization: `Bearer ${accessToken}` },
  });
  return response.data;
}

async function getNaverUserInfo(accessToken: string): Promise<NaverUserIn
  const response = await axios.get("https://openapi.naver.com/v1/nid/me",
    headers: { Authorization: `Bearer ${accessToken}` },
  });
  return response.data;
}
```

```
async function getKakaoUserInfo(accessToken: string): Promise<KakaoUserIn
  const response = await axios.get("https://kapi.kakao.com/v2/user/me", {
    headers: { Authorization: `Bearer ${accessToken}` },
  });
  return response.data;
}

async function handleSocialLogin(
  req: Request,
  res: Response,
  userInfo: SocialUserInfo
) {
  try {
    // Create or update user in database
    const openId = `${userInfo.provider}_${userInfo.id}`;
    await db.upsertUser({
      openId,
      name: userInfo.name || null,
      email: userInfo.email || null,
      loginMethod: userInfo.provider,
      lastSignedIn: new Date(),
    });

    // Create session token
    const sessionToken = await sdk.createSessionToken(openId, {
      name: userInfo.name || "",
      expiresInMs: ONE_YEAR_MS,
    });

    const cookieOptions = getSessionCookieOptions(req);
    res.cookie(COOKIE_NAME, sessionToken, { ...cookieOptions, maxAge: ONE

    res.redirect(302, "/");
  } catch (error) {
    console.error("[Social OAuth] Login failed", error);
    res.status(500).json({ error: "Social login failed" });
  }
}

export function registerSocialOAuthRoutes(app: Express) {
  // Google OAuth callback
  app.get("/api/oauth/google/callback", async (req: Request, res: Respons
    const code = getQueryParam(req, "code");
    const state = getQueryParam(req, "state");
```

```
  if (!code) {
    res.status(400).json({ error: "code is required" });
    return;
  }

  try {
    const redirectUri = `${req.protocol}://${req.get("host")}/api/oauth/

    // Exchange code for token
    const tokenResponse = await axios.post<GoogleTokenResponse>(
      "https://oauth2.googleapis.com/token",
      {
        client_id: ENV.googleClientId,
        client_secret: ENV.googleClientSecret,
        code,
        grant_type: "authorization_code",
        redirect_uri: redirectUri,
      }
    );

    // Get user info
    const googleUserInfo = await getGoogleUserInfo(tokenResponse.data.a

    const userInfo: SocialUserInfo = {
      id: googleUserInfo.sub,
      email: googleUserInfo.email,
      name: googleUserInfo.name,
      provider: "google",
    };

    await handleSocialLogin(req, res, userInfo);
  } catch (error) {
    console.error("[Google OAuth] Callback failed", error);
    res.status(500).json({ error: "Google OAuth callback failed" });
  }
});

// Naver OAuth callback
app.get("/api/oauth/naver/callback", async (req: Request, res: Response
  const code = getQueryParam(req, "code");
  const state = getQueryParam(req, "state");

  if (!code) {
```

```
    res.status(400).json({ error: "code is required" });
    return;
  }

  try {
    const redirectUri = `${req.protocol}://${req.get("host")}/api/oauth/

    // Exchange code for token
    const tokenResponse = await axios.post<NaverTokenResponse>(
      "https://nid.naver.com/oauth2.0/token",
      null,
      {
        params: {
          grant_type: "authorization_code",
          client_id: ENV.naverClientId,
          client_secret: ENV.naverClientSecret,
          code,
          state,
        },
      }
    );

    // Get user info
    const naverUserInfo = await getNaverUserInfo(tokenResponse.data.acc

    if (naverUserInfo.resultcode !== "00") {
      throw new Error("Failed to get Naver user info");
    }

    const userInfo: SocialUserInfo = {
      id: naverUserInfo.response.id,
      email: naverUserInfo.response.email,
      name: naverUserInfo.response.name || naverUserInfo.response.nickna
      provider: "naver",
    };

    await handleSocialLogin(req, res, userInfo);
  } catch (error) {
    console.error("[Naver OAuth] Callback failed", error);
    res.status(500).json({ error: "Naver OAuth callback failed" });
  }
});

// Kakao OAuth callback
```

```
app.get("/api/oauth/kakao/callback", async (req: Request, res: Response
  const code = getQueryParam(req, "code");

  if (!code) {
    res.status(400).json({ error: "code is required" });
    return;
  }

  try {
    const redirectUri = `${req.protocol}://${req.get("host")}/api/oauth/

    // Exchange code for token
    const tokenResponse = await axios.post<KakaoTokenResponse>(
      "https://kauth.kakao.com/oauth/token",
      null,
      {
        params: {
          grant_type: "authorization_code",
          client_id: ENV.kakaoClientId,
          client_secret: ENV.kakaoClientSecret,
          code,
          redirect_uri: redirectUri,
        },
        headers: {
          "Content-Type": "application/x-www-form-urlencoded",
        },
      }
    );

    // Get user info
    const kakaoUserInfo = await getKakaoUserInfo(tokenResponse.data.acc

    const userInfo: SocialUserInfo = {
      id: String(kakaoUserInfo.id),
      email: kakaoUserInfo.kakao_account.email,
      name: kakaoUserInfo.kakao_account.name || kakaoUserInfo.kakao_acc
      provider: "kakao",
    };

    await handleSocialLogin(req, res, userInfo);
  } catch (error) {
    console.error("[Kakao OAuth] Callback failed", error);
    res.status(500).json({ error: "Kakao OAuth callback failed" });
  }
```

```
  });
}
```

파일: server/_core/env.ts

```
export const ENV = {
  appId: process.env.VITE_APP_ID ?? "",
  cookieSecret: process.env.JWT_SECRET ?? "",
  databaseUrl: process.env.DATABASE_URL ?? "",
  oAuthServerUrl: process.env.OAUTH_SERVER_URL ?? "",
  ownerOpenId: process.env.OWNER_OPEN_ID ?? "",
  isProduction: process.env.NODE_ENV === "production",
  forgeApiUrl: process.env.BUILT_IN_FORGE_API_URL ?? "",
  forgeApiKey: process.env.BUILT_IN_FORGE_API_KEY ?? "",
  // Social login
  googleClientId: process.env.GOOGLE_CLIENT_ID ?? "",
  googleClientSecret: process.env.GOOGLE_CLIENT_SECRET ?? "",
  naverClientId: process.env.NAVER_CLIENT_ID ?? "",
  naverClientSecret: process.env.NAVER_CLIENT_SECRET ?? "",
  kakaoClientId: process.env.KAKAO_CLIENT_ID ?? "",
  kakaoClientSecret: process.env.KAKAO_CLIENT_SECRET ?? "",
};
```

파일: server/routers.ts

```
import { COOKIE_NAME } from "@shared/const";
import { z } from "zod";
import { getSessionCookieOptions } from "./_core/cookies";
import { systemRouter } from "./_core/systemRouter";
import { publicProcedure, router } from "./_core/trpc";
import { analyzeData, generateAnomalyData, generateNormalData, generateCa
import { clearAnomalyLogs, getRecentAnomalyLogs, insertAnomalyLog, increm
import { getRiskLevel } from "../shared/semiguard";
import { invokeLLM } from "./_core/llm";
import type { RiskLevel } from "../shared/semiguard";

export const appRouter = router({
  system: systemRouter,
  auth: router({
    me: publicProcedure.query(opts => opts.ctx.user),
    logout: publicProcedure.mutation(({ ctx }) => {
      const cookieOptions = getSessionCookieOptions(ctx.req);
      ctx.res.clearCookie(COOKIE_NAME, { ...cookieOptions, maxAge: -1 });
      return { success: true } as const;
    }),
  }),

  semiguard: router({
    injectNormal: publicProcedure.mutation(async () => {
    const dbThresholds = await getThresholds();
    const data = generateNormalData();
    const result = analyzeData(data);
    const riskLevel = getRiskLevel(result.anomalyScore, dbThresholds) as
    await incrementSampleCount();
    await insertAnomalyLog({
      userId: 1, // 기본 사용자 ID (나중에 ctx.user.id로 변경)
      current: data.current,
      temperature: data.temperature,
      vibration: data.vibration,
      noise: data.noise,
      anomalyScore: result.anomalyScore,
      riskLevel,
      isAnomaly: riskLevel === "danger" ? 1 : 0,
    });
    return { ...result, riskLevel, isAnomaly: riskLevel === "danger" };
  }),
```

```
  injectAnomaly: publicProcedure.mutation(async () => {
    const dbThresholds = await getThresholds();
    const data = generateAnomalyData();
    const result = analyzeData(data);
    const riskLevel = getRiskLevel(result.anomalyScore, dbThresholds) as
    await incrementSampleCount();
    await insertAnomalyLog({
      userId: 1, // 기본 사용자 ID (나중에 ctx.user.id로 변경)
      current: data.current,
      temperature: data.temperature,
      vibration: data.vibration,
      noise: data.noise,
      anomalyScore: result.anomalyScore,
      riskLevel,
      isAnomaly: riskLevel === "danger" ? 1 : 0,
    });
    return { ...result, riskLevel, isAnomaly: riskLevel === "danger" };
  }),

  injectCaution: publicProcedure.mutation(async () => {
    const dbThresholds = await getThresholds();
    const data = generateCautionData();
    const result = analyzeData(data);
    const riskLevel = getRiskLevel(result.anomalyScore, dbThresholds) as
    await incrementSampleCount();
    await insertAnomalyLog({
      userId: 1, // 기본 사용자 ID (나중에 ctx.user.id로 변경)
      current: data.current,
      temperature: data.temperature,
      vibration: data.vibration,
      noise: data.noise,
      anomalyScore: result.anomalyScore,
      riskLevel,
      isAnomaly: riskLevel === "danger" ? 1 : 0,
    });
    return { ...result, riskLevel, isAnomaly: riskLevel === "danger" };
  }),

  injectWarning: publicProcedure.mutation(async () => {
    const dbThresholds = await getThresholds();
    const data = generateWarningData();
    const result = analyzeData(data);
    const riskLevel = getRiskLevel(result.anomalyScore, dbThresholds) as
```

```
    await incrementSampleCount();
    await insertAnomalyLog({
      userId: 1, // 기본 사용자 ID (나중에 ctx.user.id로 변경)
      current: data.current,
      temperature: data.temperature,
      vibration: data.vibration,
      noise: data.noise,
      anomalyScore: result.anomalyScore,
      riskLevel,
      isAnomaly: riskLevel === "danger" ? 1 : 0,
    });
    return { ...result, riskLevel, isAnomaly: riskLevel === "danger" };
  }),

  autoFetch: publicProcedure.mutation(async () => {
    // 자동 폴링: 80% 정상, 10% 약한 주의, 10% 약한 경고
    const roll = Math.random();
    const data = roll < 0.80
      ? generateNormalData()
      : roll < 0.90
        ? generateSlightCautionData()
        : generateSlightWarningData();
    const dbThresholds = await getThresholds();
    const result = analyzeData(data);
    const riskLevel = getRiskLevel(result.anomalyScore, dbThresholds) as
    await incrementSampleCount();
    await insertAnomalyLog({
      userId: 1, // 기본 사용자 ID (나중에 ctx.user.id로 변경)
      current: data.current,
      temperature: data.temperature,
      vibration: data.vibration,
      noise: data.noise,
      anomalyScore: result.anomalyScore,
      riskLevel,
      isAnomaly: riskLevel === "danger" ? 1 : 0,
    });
    return { ...result, riskLevel, isAnomaly: riskLevel === "danger" };
  }),

  getLogs: publicProcedure
    .input(z.object({ limit: z.number().optional().default(50) }))
    .query(async ({ input }) => {
      const logs = await getRecentAnomalyLogs(input.limit);
        return logs.map(l => ({
```

```
        id: l.id,
        timestamp: l.timestamp.toISOString(),
        current: l.current,
        temperature: l.temperature,
        vibration: l.vibration,
        noise: l.noise,
        anomalyScore: l.anomalyScore,
        riskLevel: l.riskLevel,
        isAnomaly: l.isAnomaly === 1,
        llmAnalysisKo: l.llmAnalysisKo ?? null,
        llmAnalysisEn: l.llmAnalysisEn ?? null,
        llmAnalysisJa: l.llmAnalysisJa ?? null,
      }));
    }),

  clearLogs: publicProcedure.mutation(async () => {
  await clearAnomalyLogs();
  // 로그 초기화 시 danger_reset_offset도 0으로 리셋
  await resetSavedCost(0);
  return { success: true };
}),

  // 방문자 카운터 증가 및 조회
  trackVisit: publicProcedure.mutation(async () => {
    const todayCount = await incrementVisitor();
    const totalCount = await getTotalVisitors();
    return { todayCount, totalCount };
  }),

  getStats: publicProcedure.query(async () => {
    const [stats, totalVisitors] = await Promise.all([
      getAnomalyStats(),
      getTotalVisitors(),
    ]);
    const [totalSamples, dangerOffset] = await Promise.all([
      getTotalSamples(),
      getDangerResetOffset(),
    ]);
    const uptimePct = totalSamples > 0
      ? Math.round(((totalSamples - stats.anomalyCount) / totalSamples)
      : 100;
    // 절감 비용: 위험 단계 탐지 1회 = 약 5천만 원 절감
    // dangerOffset은 리셋 시점의 dangerCount이므로, 그 이후 새로 증가한 건수만 ㅊ
    const effectiveDanger = Math.max(0, stats.dangerCount - dangerOffse
```

```
    const savedCost = effectiveDanger * 50_000_000;
    return {
      totalDetections: stats.total,
      dangerCount: stats.dangerCount,
      anomalyCount: stats.anomalyCount,
      uptimePct,
      savedCost,
      totalVisitors,
    };
  }),

  resetSavedCost: publicProcedure.mutation(async () => {
    const stats = await getAnomalyStats();
    await resetSavedCost(stats.dangerCount);
    return { success: true };
  }),

  getDailyMaxRisk: publicProcedure.query(async () => {
    return getDailyMaxRisk();
  }),

  getThresholds: publicProcedure.query(async () => {
    return getThresholds();
  }),

  saveThresholds: publicProcedure
    .input(z.object({
      normal: z.number().int().min(1).max(98),
      caution: z.number().int().min(2).max(98),
      warning: z.number().int().min(3).max(98),
    }))
    .mutation(async ({ input }) => {
      await saveThresholds(input.normal, input.caution, input.warning);
      return { success: true };
    }),

  getRecentScores: publicProcedure
    .input(z.object({ limit: z.number().optional().default(50) }))
    .query(async ({ input }) => {
      const rows = await getRecentScores(input.limit);
      return rows.map(r => ({
        timestamp: r.timestamp.toISOString(),
        score: r.score,
        riskLevel: r.riskLevel,
```

```
    }));
  }),
getSensorThresholds: publicProcedure.query(async () => {
  return await getSensorThresholds();
}),
saveSensorThresholds: publicProcedure
  .input(z.object({
    currentCaution: z.number(), currentWarning: z.number(), currentDa
    tempCaution: z.number(), tempWarning: z.number(), tempDanger: z.nu
    vibCaution: z.number(), vibWarning: z.number(), vibDanger: z.numb
    noiseCaution: z.number(), noiseWarning: z.number(), noiseDanger: z
  }))
  .mutation(async ({ input }) => {
    await saveSensorThresholds(input);
    return { success: true };
  }),
analyzeAnomaly: publicProcedure
  .input(
    z.object({
      current: z.number(),
      temperature: z.number(),
      vibration: z.number(),
      noise: z.number(),
      anomalyScore: z.number(),
      riskLevel: z.string(),
      lang: z.enum(["ko", "en", "ja"]).default("ko"),
      logId: z.number().optional(),
    })
  )
  .mutation(async ({ input }) => {
    const { current, temperature, vibration, noise, anomalyScore, risk

    const riskLabelKo = riskLevel === "danger" ? "위험" : riskLevel ===
    const riskLabelJa = riskLevel === "danger" ? "危険" : riskLevel ===
    const riskLabelEn = riskLevel;

    const makeMessages = (systemPrompt: string, userPrompt: string) =>
      { role: "system" as const, content: systemPrompt },
      { role: "user" as const, content: userPrompt },
    ];

    const jsonSchema = {
      type: "json_schema" as const,
      json_schema: {
```

```
      name: "anomaly_analysis",
      strict: true,
      schema: {
        type: "object",
        properties: {
          primaryCause: { type: "string" },
          details: { type: "string" },
          recommendation: { type: "string" },
        },
        required: ["primaryCause", "details", "recommendation"],
        additionalProperties: false,
      },
    },
  };

  const callLlm = async (sys: string, usr: string) => {
    const res = await invokeLLM({ messages: makeMessages(sys, usr),
    const c = res.choices[0]?.message?.content;
    if (typeof c !== "string") throw new Error("no content");
    return JSON.parse(c) as { primaryCause: string; details: string
  };

  // 3개 언어 동시 LLM 호출
  const [koResult, enResult, jaResult] = await Promise.allSettled([
    callLlm(
      `당신은 반도체 공정 설비 이상 탐지 전문 AI입니다. 센서 데이터를 분석하여 이
      `센서 데이터: 전류 ${current.toFixed(2)}A, 온도 ${temperature.toFi
    ),
    callLlm(
      `You are an AI specialist in semiconductor process equipment a
      `Sensor data: Current ${current.toFixed(2)}A, Temperature ${t
    ),
    callLlm(
      `あなたは半導体プロセス設備の異常検知専門AIです。センサーデータを分析し、
      `センサーデータ: 電流${current.toFixed(2)}A、温度${temperature.to
    ),
  ]);

  const koData = koResult.status === "fulfilled" ? koResult.value :
  const enData = enResult.status === "fulfilled" ? enResult.value :
  const jaData = jaResult.status === "fulfilled" ? jaResult.value :

  // DB에 3개 언어 저장
  const targetId = logId ?? await getLastInsertedLogId();
```

```
        if (targetId) {
          await updateAnomalyLogLlm(targetId, JSON.stringify(koData), JSO
        }

        // 현재 요청 언어에 맞는 결과 반환
        return lang === "ko" ? koData : lang === "ja" ? jaData : enData;
      }),
        getLlmHistory: publicProcedure.query(async () => {
      return getLlmHistory(5);
    }),
  }),
});
export type AppRouter = typeof appRouter;
```

파일: server/db.ts

```
import { eq } from "drizzle-orm";
import { drizzle } from "drizzle-orm/mysql2";
import { InsertUser, users } from "../drizzle/schema";
import { ENV } from './_core/env';

let _db: ReturnType<typeof drizzle> | null = null;

// Lazily create the drizzle instance so local tooling can run without a
export async function getDb() {
  if (!_db && process.env.DATABASE_URL) {
    try {
      _db = drizzle(process.env.DATABASE_URL);
    } catch (error) {
      console.warn("[Database] Failed to connect:", error);
      _db = null;
    }
  }
  return _db;
}

export async function upsertUser(user: InsertUser): Promise<void> {
  if (!user.openId) {
    throw new Error("User openId is required for upsert");
  }

  const db = await getDb();
  if (!db) {
    console.warn("[Database] Cannot upsert user: database not available")
    return;
  }

  try {
    const values: InsertUser = {
      openId: user.openId,
    };
    const updateSet: Record<string, unknown> = {};

    const textFields = ["name", "email", "loginMethod"] as const;
    type TextField = (typeof textFields)[number];

    const assignNullable = (field: TextField) => {
```

```
      const value = user[field];
      if (value === undefined) return;
      const normalized = value ?? null;
      values[field] = normalized;
      updateSet[field] = normalized;
    };

    textFields.forEach(assignNullable);

    if (user.lastSignedIn !== undefined) {
      values.lastSignedIn = user.lastSignedIn;
      updateSet.lastSignedIn = user.lastSignedIn;
    }
    if (user.role !== undefined) {
      values.role = user.role;
      updateSet.role = user.role;
    } else if (user.openId === ENV.ownerOpenId) {
      values.role = 'admin';
      updateSet.role = 'admin';
    }

    if (!values.lastSignedIn) {
      values.lastSignedIn = new Date();
    }

    if (Object.keys(updateSet).length === 0) {
      updateSet.lastSignedIn = new Date();
    }

    await db.insert(users).values(values).onDuplicateKeyUpdate({
      set: updateSet,
    });
  } catch (error) {
    console.error("[Database] Failed to upsert user:", error);
    throw error;
  }
}

export async function getUserByOpenId(openId: string) {
  const db = await getDb();
  if (!db) {
    console.warn("[Database] Cannot get user: database not available");
    return undefined;
  }
```

```
  const result = await db.select().from(users).where(eq(users.openId, open

  return result.length > 0 ? result[0] : undefined;
}

// TODO: add feature queries here as your schema grows.
```

파일: server/storage.ts

```
// Preconfigured storage helpers for Manus WebDev templates
// Uploads via Forge Server presigned URL to S3 (PUT direct).
// Downloads return /manus-storage/{key} paths served via 307 redirect.

import { ENV } from "./_core/env";

function getForgeConfig() {
  const forgeUrl = ENV.forgeApiUrl;
  const forgeKey = ENV.forgeApiKey;

  if (!forgeUrl || !forgeKey) {
    throw new Error(
      "Storage config missing: set BUILT_IN_FORGE_API_URL and BUILT_IN_FOI
    );
  }

  return { forgeUrl: forgeUrl.replace(/\/+$/, ""), forgeKey };
}

function normalizeKey(relKey: string): string {
  return relKey.replace(/^\/+/, "");
}

function appendHashSuffix(relKey: string): string {
  const hash = crypto.randomUUID().replace(/-/g, "").slice(0, 8);
  const lastDot = relKey.lastIndexOf(".");
  if (lastDot === -1) return `${relKey}_${hash}`;
  return `${relKey.slice(0, lastDot)}_${hash}${relKey.slice(lastDot)}`;
}

export async function storagePut(
  relKey: string,
  data: Buffer | Uint8Array | string,
  contentType = "application/octet-stream",
): Promise<{ key: string; url: string }> {
  const { forgeUrl, forgeKey } = getForgeConfig();
  const key = appendHashSuffix(normalizeKey(relKey));

  // 1. Get presigned PUT URL from Forge
  const presignUrl = new URL("v1/storage/presign/put", forgeUrl + "/");
  presignUrl.searchParams.set("path", key);
```

```
  const presignResp = await fetch(presignUrl, {
    headers: { Authorization: `Bearer ${forgeKey}` },
  });

  if (!presignResp.ok) {
    const msg = await presignResp.text().catch(() => presignResp.statusTe
    throw new Error(`Storage presign failed (${presignResp.status}): ${ms
  }

  const { url: s3Url } = (await presignResp.json()) as { url: string };
  if (!s3Url) throw new Error("Forge returned empty presign URL");

  // 2. PUT file directly to S3
  const blob =
    typeof data === "string"
      ? new Blob([data], { type: contentType })
      : new Blob([data as any], { type: contentType });

  const uploadResp = await fetch(s3Url, {
    method: "PUT",
    headers: { "Content-Type": contentType },
    body: blob,
  });

  if (!uploadResp.ok) {
    throw new Error(`Storage upload to S3 failed (${uploadResp.status})`)
  }

  return { key, url: `/manus-storage/${key}` };
}

export async function storageGet(relKey: string): Promise<{ key: string;
  const key = normalizeKey(relKey);
  return { key, url: `/manus-storage/${key}` };
}

export async function storageGetSignedUrl(relKey: string): Promise<string
  const { forgeUrl, forgeKey } = getForgeConfig();
  const key = normalizeKey(relKey);

  const getUrl = new URL("v1/storage/presign/get", forgeUrl + "/");
  getUrl.searchParams.set("path", key);
```

```
  const resp = await fetch(getUrl, {
    headers: { Authorization: `Bearer ${forgeKey}` },
  });

  if (!resp.ok) {
    const msg = await resp.text().catch(() => resp.statusText);
    throw new Error(`Storage signed URL failed (${resp.status}): ${msg}`)
  }

  const { url } = (await resp.json()) as { url: string };
  return url;
}
```

파일: server/semiguard.ts

```
// SemiGuard AI - Isolation Forest 기반 이상탐지 엔진
import { getRiskLevel, type RiskLevel, type SensorData } from "../shared/s

// ── 정상 데이터 기준값 (학습된 기준) ──────────────────────────────
const NORMAL_BASELINE = {
  current:     { mean: 5.0,  std: 0.5 },
  temperature: { mean: 45.0, std: 3.0 },
  vibration:   { mean: 2.0,  std: 0.3 },
  noise:       { mean: 55.0, std: 4.0 },
};

// ── Isolation Forest 간소화 구현 (서버 사이드) ────────────────────
// 실제 Isolation Forest의 핵심 아이디어: 이상값일수록 더 적은 분기로 고립됨
// 여기서는 Mahalanobis 거리 기반 근사 구현 (라이브러리 없이 동작)
function computeAnomalyScore(data: SensorData): number {
  const fields: (keyof typeof NORMAL_BASELINE)[] = ["current", "temperatu
  let totalScore = 0;

  for (const field of fields) {
    const { mean, std } = NORMAL_BASELINE[field];
    const value = data[field];
    // z-score 절댓값 계산
    const z = Math.abs((value - mean) / std);
    // 각 센서의 기여도 합산 (최대 25점씩, 총 100점)
    totalScore += Math.min(z * 8, 25);
  }

  return Math.min(Math.round(totalScore), 100);
}

// ── 센서 데이터 생성기 ──────────────────────────────────────────
export function generateNormalData(): SensorData {
  const rand = (mean: number, std: number) =>
    mean + (Math.random() - 0.5) * std * 2;

  return {
    current:     parseFloat(rand(NORMAL_BASELINE.current.mean,     NORMAL
    temperature: parseFloat(rand(NORMAL_BASELINE.temperature.mean, NORMAL
    vibration:   parseFloat(rand(NORMAL_BASELINE.vibration.mean,   NORMAL
    noise:       parseFloat(rand(NORMAL_BASELINE.noise.mean,       NORMAL
    timestamp:   Date.now(),
```

```
  };
}

export function generateAnomalyData(): SensorData {
  // 위험 상태: 3개 센서만 크게 이탈, 1개는 약간 이탈 → 점수 70~95 범위
  // 4개 모두 z=3.5+ 이탈 시 항상 100이 되므로, 이탈 센서 수와 크기를 조절
  const fields: (keyof typeof NORMAL_BASELINE)[] = ["current", "temperatu
  // 3개 센서는 크게 이탈(z=2.8~3.8), 1개는 약간 이탈(z=1.0~1.5) → 총 점수 70~95
  const shuffled = [...fields].sort(() => Math.random() - 0.5);
  const bigDeviateFields = new Set(shuffled.slice(0, 3));

  const result: Record<string, number> = { timestamp: Date.now() };
  for (const field of fields) {
    const { mean, std } = NORMAL_BASELINE[field];
    const factor = bigDeviateFields.has(field)
      ? 2.8 + Math.random() * 1.0   // z=2.8~3.8 → 점수 22.4~25(cap)
      : 1.0 + Math.random() * 0.5;  // z=1.0~1.5 → 점수 8~12
    const sign = Math.random() > 0.5 ? 1 : -1;
    const val = mean + std * factor * sign;
    result[field] = parseFloat(val.toFixed(field === "temperature" || fie
  }
  return result as unknown as SensorData;
}

// 주의 단계 데이터: 점수 30~49 보장
// 4개 센서 모두 이탈 시 점수가 60+ 로 올라가므로, 2개만 중간 이탈(z=1.8~2.3), 2개는
export function generateCautionData(): SensorData {
  const fields: (keyof typeof NORMAL_BASELINE)[] = ["current", "temperatu
  const shuffled = [...fields].sort(() => Math.random() - 0.5);
  const devFields = new Set(shuffled.slice(0, 2)); // 2개만 이탈
  const result: Record<string, number> = { timestamp: Date.now() };
  for (const field of fields) {
    const { mean, std } = NORMAL_BASELINE[field];
    if (devFields.has(field)) {
      // 이탈: z=1.8~2.3 → 점수 14.4~18.4 per sensor, 2개 합산 28~37
      const factor = 1.8 + Math.random() * 0.5;
      const sign = Math.random() > 0.5 ? 1 : -1;
      const val = mean + std * factor * sign;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
    } else {
      // 정상: z=0~0.5 → 점수 0~4 per sensor
      const val = mean + (Math.random() - 0.5) * std * 0.8;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
    }
```

```
  }
  return result as unknown as SensorData;
}

// 경고 단계 데이터: 점수 50~69 보장
// 3개 센서 이탈(z=2.0~2.5), 1개 정상 → 3*16~20 + 1*2 = 50~62
export function generateWarningData(): SensorData {
  const fields: (keyof typeof NORMAL_BASELINE)[] = ["current", "temperatu
  const shuffled = [...fields].sort(() => Math.random() - 0.5);
  const devFields = new Set(shuffled.slice(0, 3)); // 3개 이탈
  const result: Record<string, number> = { timestamp: Date.now() };
  for (const field of fields) {
    const { mean, std } = NORMAL_BASELINE[field];
    if (devFields.has(field)) {
      // 이탈: z=2.0~2.5 → 점수 16~20 per sensor, 3개 합산 48~60
      const factor = 2.0 + Math.random() * 0.5;
      const sign = Math.random() > 0.5 ? 1 : -1;
      const val = mean + std * factor * sign;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
    } else {
      // 정상: z=0~0.5 → 점수 0~4
      const val = mean + (Math.random() - 0.5) * std * 0.8;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
    }
  }
  return result as unknown as SensorData;
}

// 약한 주의: 1개 센서 살짝 이탈 (점수 10~25 → 주의 단계 진입)
export function generateSlightCautionData(): SensorData {
  const fields: (keyof typeof NORMAL_BASELINE)[] = ["current", "temperatu
  const devField = fields[Math.floor(Math.random() * fields.length)];
  const result: Record<string, number> = { timestamp: Date.now() };
  for (const field of fields) {
    const { mean, std } = NORMAL_BASELINE[field];
    if (field === devField) {
      // z=1.3~1.7 → 점수 10~14 per sensor
      const factor = 1.3 + Math.random() * 0.4;
      const sign = Math.random() > 0.5 ? 1 : -1;
      const val = mean + std * factor * sign;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
    } else {
      const val = mean + (Math.random() - 0.5) * std * 0.6;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
```

```
    }
  }
  return result as unknown as SensorData;
}

// 약한 경고: 2개 센서 이탈 (점수 30~50 → 경고 단계)
export function generateSlightWarningData(): SensorData {
  const fields: (keyof typeof NORMAL_BASELINE)[] = ["current", "temperatu
  const shuffled = [...fields].sort(() => Math.random() - 0.5);
  const devFields = new Set(shuffled.slice(0, 2));
  const result: Record<string, number> = { timestamp: Date.now() };
  for (const field of fields) {
    const { mean, std } = NORMAL_BASELINE[field];
    if (devFields.has(field)) {
      // z=1.5~2.0 → 점수 12~16 per sensor, 2개 합산 24~32
      const factor = 1.5 + Math.random() * 0.5;
      const sign = Math.random() > 0.5 ? 1 : -1;
      const val = mean + std * factor * sign;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
    } else {
      const val = mean + (Math.random() - 0.5) * std * 0.6;
      result[field] = parseFloat(val.toFixed(field === "temperature" || f
    }
  }
  return result as unknown as SensorData;
}

export function analyzeData(data: SensorData) {
  const anomalyScore = computeAnomalyScore(data);
  const riskLevel: RiskLevel = getRiskLevel(anomalyScore);
  const isAnomaly = anomalyScore >= 70;
  return { sensorData: data, anomalyScore, riskLevel, isAnomaly };
}
```

파일: server/semiguardDb.ts

```
import { desc, eq, sql, count as drizzleCount } from "drizzle-orm";
import { anomalyLogs, visitorStats, sampleStats, thresholdSettings, sensor
import { getDb } from "./db";

export async function insertAnomalyLog(entry: InsertAnomalyLog) {
  const db = await getDb();
  if (!db) return null;
  await db.insert(anomalyLogs).values(entry);
}

// 특정 로그에 LLM 분석 결과 업데이트 (3개 언어)
export async function updateAnomalyLogLlm(id: number, ko: string, en: str
  const db = await getDb();
  if (!db) return;
  await db.update(anomalyLogs).set({ llmAnalysisKo: ko, llmAnalysisEn: en
}

// 가장 최근 삽입된 로그 ID 조회
export async function getLastInsertedLogId(): Promise<number | null> {
  const db = await getDb();
  if (!db) return null;
  const rows = await db.select({ id: anomalyLogs.id }).from(anomalyLogs).
  return rows[0]?.id ?? null;
}

// LLM 분석 결과가 있는 최근 로그 N건 조회 (히스토리 패널용) - 3개 언어
export async function getLlmHistory(limit = 5): Promise<{ id: number; time
  const db = await getDb();
  if (!db) return [];
  const rows = await db.select({
    id: anomalyLogs.id,
    timestamp: anomalyLogs.timestamp,
    riskLevel: anomalyLogs.riskLevel,
    anomalyScore: anomalyLogs.anomalyScore,
    llmAnalysisKo: anomalyLogs.llmAnalysisKo,
    llmAnalysisEn: anomalyLogs.llmAnalysisEn,
    llmAnalysisJa: anomalyLogs.llmAnalysisJa,
  }).from(anomalyLogs)
    .where(sql`llm_analysis_ko IS NOT NULL OR llm_analysis_en IS NOT NULL
    .orderBy(desc(anomalyLogs.timestamp))
    .limit(limit);
```

```
  return rows;
}

export async function getRecentAnomalyLogs(limit = 50) {
  const db = await getDb();
  if (!db) return [];
  return db.select().from(anomalyLogs).orderBy(desc(anomalyLogs.timestamp
}

export async function clearAnomalyLogs() {
  const db = await getDb();
  if (!db) return;
  await db.delete(anomalyLogs);
}

// 전체 샘플 카운터 증가
export async function incrementSampleCount(): Promise<void> {
  const db = await getDb();
  if (!db) return;
  await db.insert(sampleStats).values({ key: "total_samples", value: 1 })
    .onDuplicateKeyUpdate({ set: { value: sql`${sampleStats.value} + 1` }
}

// 전체 샘플 수 조회
export async function getTotalSamples(): Promise<number> {
  const db = await getDb();
  if (!db) return 0;
  const rows = await db.select().from(sampleStats).where(eq(sampleStats.k
  return Number(rows[0]?.value ?? 0);
}

// 절감 비용 리셋 오프셋 저장 (리셋 시점의 dangerCount를 저장)
export async function resetSavedCost(currentDangerCount: number): Promise<
  const db = await getDb();
  if (!db) return;
  await db.insert(sampleStats).values({ key: "danger_reset_offset", value
    .onDuplicateKeyUpdate({ set: { value: currentDangerCount } });
}

// 절감 비용 리셋 오프셋 조회
export async function getDangerResetOffset(): Promise<number> {
  const db = await getDb();
  if (!db) return 0;
  const rows = await db.select().from(sampleStats).where(eq(sampleStats.k
```

```
  return Number(rows[0]?.value ?? 0);
}

// 오늘 방문자 수 증가 (upsert)
export async function incrementVisitor(): Promise<number> {
  const db = await getDb();
  if (!db) return 0;
  const today = new Date().toISOString().slice(0, 10); // YYYY-MM-DD
  await db.insert(visitorStats).values({ date: today, count: 1 })
    .onDuplicateKeyUpdate({ set: { count: sql`${visitorStats.count} + 1` }
  const rows = await db.select().from(visitorStats).where(eq(visitorStats
  return rows[0]?.count ?? 1;
}

// 누적 방문자 수 합계
export async function getTotalVisitors(): Promise<number> {
  const db = await getDb();
  if (!db) return 0;
  const rows = await db.select({ total: sql<number>`SUM(${visitorStats.co
  return Number(rows[0]?.total ?? 0);
}

// 날짜별 최고 위험도 집계 (히트맵용)
export async function getDailyMaxRisk(): Promise<{ date: string; riskLeve
  const db = await getDb();
  if (!db) return [];
  // 완전한 raw SQL — CASE 구문을 Drizzle 컬럼 참조 없이 순수 문자열로 처리
  const rows = await db.execute<{ date: string; maxRiskOrder: number }>(
    sql`SELECT DATE(timestamp) AS date,
        MAX(CASE risk_level
          WHEN 'danger'  THEN 4
          WHEN 'warning' THEN 3
          WHEN 'caution' THEN 2
          ELSE 1
        END) AS maxRiskOrder
        FROM anomaly_logs
        GROUP BY DATE(timestamp)
        ORDER BY DATE(timestamp) ASC`
  );
  const riskMap: Record<number, string> = { 4: "danger", 3: "warning", 2:
  return rows.map((r: any) => ({
    date: String(r.date).slice(0, 10),
    riskLevel: riskMap[Number(r.maxRiskOrder)] ?? "normal",
  }));
```

```
}

// 이상 탐지 통계
export async function getAnomalyStats(): Promise<{ total: number; dangerC
  const db = await getDb();
  if (!db) return { total: 0, dangerCount: 0, anomalyCount: 0 };
  const totalRows = await db.select({ cnt: sql<number>`COUNT(*)` }).from(a
  const dangerRows = await db.select({ cnt: sql<number>`COUNT(*)` }).from
    .where(eq(anomalyLogs.riskLevel, "danger"));
  const anomalyRows = await db.select({ cnt: sql<number>`COUNT(*)` }).fro
    .where(eq(anomalyLogs.isAnomaly, 1));
  return {
    total: Number(totalRows[0]?.cnt ?? 0),
    dangerCount: Number(dangerRows[0]?.cnt ?? 0),
    anomalyCount: Number(anomalyRows[0]?.cnt ?? 0),
  };
}
// 위험도 임계값 불러오기 (없으면 기본값 반환)
export async function getThresholds(): Promise<{ normal: number; caution:
  const db = await getDb();
  if (!db) return { normal: 29, caution: 49, warning: 69 };
  const rows = await db.select().from(thresholdSettings).where(eq(thresho
  if (!rows[0]) return { normal: 29, caution: 49, warning: 69 };
  return { normal: rows[0].normalMax, caution: rows[0].cautionMax, warnin
}

// 위험도 임계값 저장 (upsert)
export async function saveThresholds(normal: number, caution: number, war
  const db = await getDb();
  if (!db) return;
  await db.insert(thresholdSettings)
    .values({ key: "default", normalMax: normal, cautionMax: caution, war
    .onDuplicateKeyUpdate({ set: { normalMax: normal, cautionMax: caution
}

// 위험도 추이 (최근 N개 점수, 시간 오름차순)
export async function getRecentScores(limit = 50): Promise<{ timestamp: Da
  const db = await getDb();
  if (!db) return [];
  const rows = await db.select({
    timestamp: anomalyLogs.timestamp,
    score: anomalyLogs.anomalyScore,
    riskLevel: anomalyLogs.riskLevel,
  }).from(anomalyLogs).orderBy(desc(anomalyLogs.timestamp)).limit(limit);
```

```
  return rows.reverse(); // 시간 오름차순으로 반환
}

// 이상 탐지 통계
// 센서별 임계값 기본값
const SENSOR_THRESHOLD_DEFAULTS = {
  currentCaution: 7.0, currentWarning: 9.0, currentDanger: 11.0,
  tempCaution: 55.0, tempWarning: 70.0, tempDanger: 85.0,
  vibCaution: 2.3, vibWarning: 2.6, vibDanger: 3.0,
  noiseCaution: 65.0, noiseWarning: 75.0, noiseDanger: 85.0,
};

export type SensorThresholdValues = typeof SENSOR_THRESHOLD_DEFAULTS;

// 센서별 임계값 불러오기
export async function getSensorThresholds(): Promise<SensorThresholdValue
  const db = await getDb();
  if (!db) return SENSOR_THRESHOLD_DEFAULTS;
  const rows = await db.select().from(sensorThresholds).where(eq(sensorTh
  if (!rows[0]) return SENSOR_THRESHOLD_DEFAULTS;
  const r = rows[0];
  return {
    currentCaution: r.currentCaution, currentWarning: r.currentWarning, cu
    tempCaution: r.tempCaution, tempWarning: r.tempWarning, tempDanger: r
    vibCaution: r.vibCaution, vibWarning: r.vibWarning, vibDanger: r.vibDa
    noiseCaution: r.noiseCaution, noiseWarning: r.noiseWarning, noiseDange
  };
}

// 센서별 임계값 저장 (upsert)
export async function saveSensorThresholds(values: SensorThresholdValues)
  const db = await getDb();
  if (!db) return;
  await db.insert(sensorThresholds)
    .values({ key: "default", ...values })
    .onDuplicateKeyUpdate({ set: values });
}
```